

Lists I — Sequential Data

Lecture 17

CS 1001

Kirk Duran

Today

- ▶ Motivate lists as one structure for representing related data.
- ▶ Define lists as ordered, indexed, mutable sequences of elements.
- ▶ Construct lists with list literals, including the empty list and mixed-type lists.
- ▶ Access elements using zero-based positive and negative indexes.
- ▶ Test membership and locate an element with `.index()`.
- ▶ Mutate one element by assigning through an index.
- ▶ Build a useful memory model, then refine it to match how Python lists are actually represented.

Key idea

A list gives one name to an ordered sequence of element positions. Each position can be read or updated by its index.

Part 1: Why Sequential Data?

Related Data Is Awkward When Every Value Has Its Own Name

Separate variables

```
test1 = 90  
test2 = 88  
test3 = 76  
test4 = 92
```

- ▶ The values have the same conceptual role: test scores.
- ▶ Their relationship exists only in the programmer's naming convention.
- ▶ Adding more related values requires inventing more names.

Image Placeholder

`related-data-variables-vs-list.png`

A List Groups Related Values Under One Name

One sequence of scores

```
scores = [90, 88, 76, 92]
```

- ▶ `scores` is one variable name.
- ▶ The value bound to `scores` is a list containing four elements.
- ▶ Each element occupies a specific position in the sequence.

Key idea

The list represents the relationship explicitly: these values belong to one ordered sequence.

Definition: Python List

Definition

A **list** is an ordered, indexed, mutable sequence of zero or more elements.

Property	Meaning
ordered	element position is part of the list's structure
indexed	a position can be addressed by an integer index
mutable	an existing element can be replaced after creation
sequence	multiple elements are represented in a defined order

Order Is Part of the List Value

List A

[10, 20, 30]

List B

[30, 20, 10]

- ▶ The lists contain the same three integer values.
- ▶ They do not represent the same sequence because the positions differ.
- ▶ Consequently, `[10, 20, 30] == [30, 20, 10]` evaluates to `False`.

Key idea

In sequential data, position carries information.

Part 2: Creating Lists

List Literals Use Square Brackets

General form

```
[element0, element1, element2, ...]
```

- ▶ Square brackets delimit the list literal.
- ▶ Commas separate element expressions.
- ▶ Each element expression is evaluated and becomes one element of the new list.

Examples

```
scores = [90, 88, 76]
```

```
names = ["Ada", "Grace", "Alan"]
```

```
flags = [True, False, True]
```

```
empty = []    zero elements and no valid element index
```

Python Lists May Contain Mixed Types

Heterogeneous list

```
record = [42, "hello", True, 3.5]
```

- ▶ The elements do not need to share one Python type.
- ▶ Position remains well-defined even when the element types differ.
- ▶ Mixed types are legal; whether they are a clear design choice depends on what the positions represent.

Key idea

Python constrains the structure of the sequence, not the type of every value stored in that sequence.

Our Working Model: Adjacent Numbered Slots

Image Placeholder

`list-adjacent-slots-model.png`

- ▶ Think of a list as a row of adjacent element slots.
- ▶ Each slot has a unique integer position called its index.
- ▶ This model makes access and mutation easy to reason about.

Modeling note

This is our conceptual model for tracing. We will refine the physical memory representation at

Part 3: Indexing

An Index Identifies One Position in a Sequence

Definition

An **index** is an integer used to identify a particular element position in a sequence.

Example list

```
fruits = ["apple", "banana", "cherry", "date"]
```

element	apple	banana	cherry	date
index	0	1	2	3

Python Uses Zero-Based Indexing

- ▶ The first element is at index 0, not index 1.
- ▶ The second element is at index 1.
- ▶ For a list containing n elements, the last positive index is $n - 1$.

Example

```
fruits = ["apple", "banana", "cherry", "date"]
```

```
fruits[0] → "apple"
```

```
fruits[1] → "banana"
```

```
fruits[3] → "date"
```

Indexing Is an Expression

Expression

`fruits[2]`

1. Evaluate the name `fruits` to obtain the list.
2. Evaluate the index expression `2`.
3. Select the element at position `2`.
4. The complete indexing expression evaluates to that element's value.

Key idea

Just like arithmetic or a function call, an indexing expression produces a value and can participate in a larger expression, such as `scores[1] + 5`.

Trace: Reading an Element Does Not Change the List

Code

```
scores = [90, 88, 76]
middle = scores[1]
result = middle + 2
```

Statement	Expression result	Bindings afterward
scores = [90,88,76]	list value	scores → [90,88,76]
middle = scores[1]	88	middle → 88
result = middle + 2	90	result → 90

Key idea

Accessing an element retrieves a value; it does not mutate the list.

An Index Must Refer to an Existing Position

List

```
scores = [90, 88, 76]
```

Valid

```
scores[0]
```

```
scores[1]
```

```
scores[2]
```

Invalid

```
scores[3]
```

```
raises IndexError
```

Key idea

The number of elements is three, but the valid positive indexes are 0, 1, and 2.

Negative Indexes Count Backward from the End

List

```
fruits = ["apple", "banana", "cherry", "date"]
```

Image Placeholder

positive-negative-indexing.png

Positive and Negative Indexes Can Name the Same Position

Element	Positive index	Negative index
apple	0	-4
banana	1	-3
cherry	2	-2
date	3	-1

Key idea

For a list containing n elements, a valid negative index i identifies the same position as $n + i$.

Example

With four elements, index -2 corresponds to $4 + (-2) = 2$. The valid negative indexes are -1 through -4 ; -5 raises `IndexError`.

Prediction: Resolve Each Index Before Running Python

Given

```
values = [12, 24, 36, 48, 60]
```

In class

Determine the value or error produced by each expression:

1. `values[0]`
2. `values[3]`
3. `values[-1]`
4. `values[-5]`
5. `values[5]`

Part 4: Membership and Finding Positions

Membership Asks Whether a Value Is Present

Definition

A **membership test** asks whether a value occurs as an element of a sequence and produces a Boolean result.

Examples

```
fruits = ["apple", "banana", "cherry"]
```

```
"banana" in fruits → True
```

```
"orange" in fruits → False
```

```
"orange" not in fruits → True
```

Key idea

Membership answers an existence question; it does not produce the element's position.

Trace: Membership Produces a Boolean Value

Code

```
colors = ["red", "green", "blue"]
target = "green"
found = target in colors
```

Expression	Value
target	"green"
colors	["red", "green", "blue"]
target in colors	True

Key idea

The final binding is found → True; the list itself is unchanged.

`.index()` Asks Where a Value Occurs

Example

```
colors = ["red", "green", "blue"]
```

```
colors.index("green") → 1
```

Definition

The list method `.index(value)` returns the index of the first element equal to `value`.

- ▶ If a value occurs more than once, `.index()` returns the first matching position.
- ▶ Example: `[90, 75, 88, 75].index(75) → 1`.

Key idea

Indexing uses a known position to obtain a value; `.index()` uses a known value to obtain a position.

`.index()` Raises an Error When the Value Is Absent

Example

```
colors = ["red", "green", "blue"]  
colors.index("orange")
```

Result

Python raises `ValueError` because no matching element exists in the list.

Key idea

Use a membership test when the immediate question is whether the value exists. Use `.index()` when the required result is its position.

Membership and `.index()` Answer Different Questions

Image Placeholder

membership-versus-index.png

Question	Operation	Result type
Is it present?	<code>value in items</code>	<code>bool</code>
Where is it?	<code>items.index(value)</code>	<code>int</code>

Part 5: Mutating Individual Elements

Lists Are Mutable

Definition

A value is **mutable** when its state can be changed after the value has been created.

Before

```
scores = [90, 88, 76]
```

Indexed assignment

```
scores[1] = 91
```

After

```
scores → [90, 91, 76]
```

Indexed Assignment Replaces One Existing Element

1. Evaluate the list expression.
2. Evaluate the index expression.
3. Evaluate the right-hand-side expression.
4. Replace the element at the selected position with the resulting value.

Image Placeholder

`indexed-mutation.png`

Reading and Writing Use Similar Syntax but Different Semantics

Read

```
x = scores[1]
```

scores[1] is an expression.
It produces the current element value.

Write

```
scores[1] = 91
```

scores[1] is an assignment target.
It identifies the position to replace.

Key idea

Context determines whether bracket notation retrieves an element or identifies where assignment should store a new value.

Invalid Mutation Targets Raise `IndexError`

Example

```
items = [10, 20, 30]
```

```
items[3] = 40
```

Why this fails

Index 3 does not identify an existing element position. Indexed assignment replaces one existing element; it neither adds a new position nor changes the number of positions in the list.

Trace: List State Can Change Across Statements

Code

```
values = [5, 10, 15]
x = values[1]
values[1] = x + 2
last = values[-1]
```

After statement	Relevant scalar binding	List state
list creation	—	[5,10,15]
x = values[1]	x = 10	[5,10,15]
indexed assignment	x = 10	[5,12,15]
last = values[-1]	last = 15	[5,12,15]

Prediction: Track Both Position and State

Code

```
data = [8, 6, 7, 5]
data[0] = data[-1]
middle = data[2]
data[-2] = 9
```

In class

Without running Python, determine:

1. the list after the first mutation;
2. the value bound to `middle`;
3. the final state of `data`.

Part 6: From the Conceptual Model to Physical Memory



Why the Adjacent-Slot Model Is Useful

Image Placeholder

`indexed-slot-address-model.png`

- ▶ A numbered row of fixed-width slots provides a direct way to locate a position.
- ▶ Conceptually, position i is found from the beginning of the row plus an offset for i slots.
- ▶ This explains why an index identifies one element position rather than searching by a

Many Low-Level Arrays Store Homogeneous Values

- ▶ In many programming languages, a conventional array stores elements of one declared type.
- ▶ A homogeneous element type gives every array slot a predictable representation and size.
- ▶ Adjacent fixed-size elements therefore support direct address calculation from an index.

Conceptual homogeneous array

[10] [20] [30] [40] all slots store the same kind of value

Key idea

Python preserves the indexed array idea but changes what each physical slot stores.

A Python List Stores Adjacent References, Not Adjacent Objects

- ▶ In CPython, the list maintains a dynamically allocated contiguous array of object references.
- ▶ Each physical slot stores a reference to a Python object.
- ▶ The referenced objects themselves may reside elsewhere in memory.

Key idea

What is physically adjacent is the array of references.

Image Placeholder

`python-list-references.png`

References Explain Why Mixed-Type Lists Work

Python list

```
record = [42, "hello", True, 3.5]
```

- ▶ The integer, string, Boolean, and floating-point objects have different representations.
- ▶ The list slots do not need to store those object representations directly.
- ▶ Each slot stores the same kind of thing: a reference to a Python object.

Key idea

Uniform reference slots allow one Python list to refer to objects of different types.

Keep the Simpler Model for Ordinary Tracing

Tracing model

[90] [88] [76]

Think of indexed values in adjacent numbered positions.

Implementation model

adjacent reference slots

↓ ↓ ↓

Python objects elsewhere in memory

Key idea

Use the simplest model that correctly predicts the behavior being studied. Refine the model when implementation details become relevant.

Lecture 17 Mental Model

Concept	Operational question
list	What ordered sequence of elements is represented?
index	Which position is being addressed?
indexing expression	What value occupies that position?
membership	Does this value occur in the sequence?
<code>.index()</code>	What is the first matching position?
indexed assignment	What existing position is being replaced?

Takeaways

- ▶ A list is an ordered, indexed, mutable sequence.
- ▶ List literals use square brackets; `[]` represents an empty list.
- ▶ Positive indexes begin at 0; negative indexes count backward from the end.
- ▶ Invalid element positions raise `IndexError`.
- ▶ `in` and `not in` answer membership questions with Boolean values.
- ▶ `.index(value)` returns the first matching position and raises `ValueError` when no match exists.
- ▶ Indexed assignment mutates one existing element while preserving the sequence's positions.
- ▶ For tracing, think in adjacent numbered positions; physically, CPython stores an adjacent array of references to objects.